



Ecole Nationale de l'Aviation Civile
7 Av. Edouard Belin
31400 Toulouse Cedex 9



**Institut de Recherche en Informatique de
Toulouse**
Cr Rose Dieng-Kuntz
31400 Toulouse



LAAS-CNRS
7 Avenue du Colonel Roche
31400 Toulouse



**Artificial and Natural Intelligence Toulouse
Institute**
3 Rue Tarfaya
31400 Toulouse

Amélioration de Méthodes de Benders pour un Problème de Planification Robuste sous Incertitude

Mathis FRANCINE-HABAS

Encadrants : Christian Artigues (Directeur de Recherche au LAAS-CNRS)
Romain Guillaume (Enseignant-Chercheur à l'IRIT/ANITI)

Tuteur Universitaire : Mohammed Sbihi (Enseignant-Chercheur à l'ENAC)

Soutenu le : 31 / 08 / 2026

2025-2026

Remerciements

Je tiens à remercier toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce stage et à la rédaction de ce rapport.

Je remercie tout particulièrement Messieurs Christian Artigues et Romain Guillaume pour la confiance qu'ils m'ont accordée ainsi que pour leur aide tout au long de ce projet de recherche.

Je souhaite également remercier l'ensemble de mes professeurs pour la qualité de leur enseignement, qui m'ont permis d'acquérir les connaissances nécessaires à la réussite de ma formation et de ce stage.

Mathis FRANCINE-HABAS

Table des matières

I	Contexte de la recherche	3
II	Littérature	4
III	Présentation du travail effectué	5
III.I	Objectifs	5
III.II	Propositions	5
III.III	Résultats	5
IV	Conclusion	6
IV.I	Synthèse sur le travail effectué	6
IV.II	Apports professionnels	6
IV.III	Apports techniques	6
V	Annexes	7
A	Méthodologie et Implémentation	8
I	Modélisation Mathématique	8
II	Pseudo-code de l'Algorithme	8
III	Implémentation en C++	9

Table des figures

Liste des tableaux

List of Acronyms

Introduction

Historiquement issue des stratégies militaires de la Seconde Guerre mondiale, la Recherche Opérationnelle (RO) s'est depuis imposée comme la discipline scientifique incontournable pour l'allocation optimale des ressources et l'amélioration des processus industriels. Son efficacité actuelle repose sur un trypique fondamental. Le premier axe consiste à modéliser mathématiquement ces processus complexes afin d'en traduire fidèlement les contraintes et les objectifs. Le deuxième réside dans le développement de méthodes de résolution capables de passer à l'échelle industrielle. Enfin, le dernier levier de performance, devenu central aujourd'hui, est le traitement intégré et intelligent des données.

C'est précisément sur ce dernier point que la Recherche Opérationnelle converge vers l'Intelligence Artificielle. Cependant, l'exploitation de ces données issus d'environnements réels confronte systématiquement les modèles à un défi majeur : l'omniprésence de l'incertitude.

En effet, l'instabilité de la demande, la volatilité des prix ou encore les aléas climatiques complexifient la planification au quotidien. La production, la logistique et les approvisionnements subissent des risques accrus, largement amplifiés par l'interdépendance des chaînes mondiales. Les modèles d'optimisation classiques supposent souvent un monde déterministe où les paramètres sont connus à l'avance. Or, une stratégie optimisée pour ce scénario "parfait" s'avère très fragile et peut devenir irréalisable au moindre aléa. L'enjeu n'est donc plus seulement d'optimiser un plan idéal, mais de concevoir des solutions capables d'absorber ces écarts : c'est la naissance de l'optimisation robuste.

L'optimisation robuste pose un cadre formel pour décider dans l'incertain. Elle cherche à identifier la meilleure décision qui reste réalisable et performante face à toutes les incertitudes futures, quels que soient les scénarios qui se concrétisent. Ces incertitudes, qu'elles soient continues (ex. variations de température) ou discrètes (ex. scénarios de pannes), s'appuient sur des historiques, des retours métiers ou des scénarios critiques générés par l'IA. Les applications sont cruciales, notamment dans la supply chain ou la production, où les aléas locaux peuvent déclencher des effets sur la performance globale.

Cependant, cette garantie de robustes a un coût computationnel considérable. La prise en compte exhaustive des incertitudes entraîne une explosion de la taille des modèles mathématiques (notamment en Programmation Linéaire en Nombre Entiers). Les solveurs traditionnels se retrouvent alors rapidement limités par la complexité temporelle et spatiale de ces problèmes.

Dès lors, comment résoudre efficacement ces modèles robustes à grande échelle ? C'est pour franchir ce mur calculatoire que les méthodes de décomposition s'avèrent indispensables. Parmi elles, la décomposition de Benders se distingue en séparant les décisions stratégiques (le problème Maître) des évaluations de scénarios (les sous-problèmes). La problématique centrale de ce stage a donc été d'étudier et d'améliorer

les méthodes de Benders afin d'accélérer la résolution de problèmes de planification robuste sous incertitude.

Pour répondre à cette problématique, ce rapport s'articule autour de quatre chapitres. Le premier posera le contexte institutionnel et scientifique de la recherche. Le deuxième dressera l'état de l'art des approches d'optimisation robuste et détaillera le fonctionnement classique de la décomposition de Benders. Ensuite, nous nous consacrerons aux contributions méthodologiques et aux améliorations algorithmiques proposées durant cette recherche. Enfin, nous détaillerons l'implémentation informatique et analyserons les résultats expérimentaux obtenus, avant de conclure sur les apports techniques et professionnels de cette expérience.

Comment les méthodes de benders peuvent optimiser la planification robuste sous incertitude? Nous ferons tout d'abord un point sur les avancées scientifiques dans ce domaine, puis nous présenterons le contexte de la recherche, la recherche en elle-même, ses résultats ainsi que les apports techniques et professionnels de cette expérience.

I Contexte de la recherche

Ce stage s'inscrit au cœur d'un écosystème scientifique toulousain riche, structuré autour d'une collaboration entre trois acteurs majeurs de la recherche en intelligence artificielle et en informatique : l'institut ANITI, l'IRIT et le LAAS-CNRS.

L'impulsion de cette dynamique est notamment portée par ANITI (Artificial and Natural Intelligence Toulouse Institute), un cluster labellisé par la Communauté d'universités et d'établissements de Toulouse. Son ambition est de concevoir des systèmes d'Intelligence Artificielle dont les prises de décision sont explicables, équitables, fiables et robustes, une condition indispensable pour permettre leur déploiement dans des secteurs critiques à haut risque.

Cette volonté de développer des systèmes décisionnels de confiance s'appuie sur l'expertise de deux laboratoires. D'une part, l'Institut de Recherche en Informatique de Toulouse (IRIT), une unité de recherche publique placée sous la tutelle conjointe du Centre National de la Recherche Scientifique (CNRS) et des universités toulousaines. D'autre part, le Laboratoire d'Analyse et d'Architecture des Systèmes (LAAS) est une unité propre du CNRS. Si le LAAS aborde des thématiques très variées, allant de la robotique aux micro-nano-bio-technologies, il se distingue particulièrement par sa recherche théorique et méthodologique sur la conception de lois mathématiques appliquées à la commande et à la décision.

C'est à l'intersection de ces institutions, et plus spécifiquement au sein de l'équipe ROC (Recherche sur des problèmes d'Optimisation Combinatoire) du LAAS, que s'est déroulé mon stage. Positionnée à la frontière entre la Recherche Opérationnelle et l'Intelligence Artificielle, cette équipe offre un cadre scientifique idéal pour aborder les problématiques d'optimisation robuste qui font l'objet de ce rapport.

IMPORTANT : parler de l'utilisation à terme du modèle par AIRBUS.

II Litterature

TODO : STATE OF THE ART

III Présentation du travail effectué

III.I Objectifs

III.II Propositions

III.III Résultats

IV Conclusion

IV.I Synthèse sur le travail effectué

IV.II Apports professionnels

IV.III Apports techniques

V Annexes

Annexe A

Méthodologie et Implémentation

I Modélisation Mathématique

Voici comment introduire les concepts formels de manière élégante. Les environnements sont numérotés automatiquement en fonction du chapitre.

Définition A.1: Problème Maître Restreint (RMP)

Dans le cadre de la décomposition de Benders, le Problème Maître Restreint contient un sous-ensemble des contraintes du problème original. Il coordonne la recherche en proposant une solution partielle qui sera évaluée par le sous-problème.

Théorème A.1: Convergence de la décomposition

Si le domaine des solutions réalisables est borné et non vide, l'algorithme converge en un nombre fini d'itérations vers l'optimum global du problème de planification.

Preuve. À chaque itération, soit la solution actuelle est optimale, soit une nouvelle coupe (d'optimalité ou de faisabilité) est ajoutée au problème maître. L'espace des coupes possibles étant fini pour un problème linéaire en nombres entiers, l'algorithme ne peut cycler indéfiniment. ■

Propriété A.1: Structure des coupes

Les coupes générées agissent comme des minorants stricts de la fonction objectif dans la région de l'espace explorée.

II Pseudo-code de l'Algorithme

L'environnement `algorithm2e` est le standard pour les articles scientifiques. Il génère des blocs clairs avec des lignes de structure.

Algorithme 1 : Décomposition de Benders classique

Données : Modèle MILP initial, Tolérance ε
Résultat : Solution optimale x^*, y^*
 Initialiser le RMP sans coupes;
 $UB \leftarrow +\infty$;
 $LB \leftarrow -\infty$;
while $(UB - LB) > \varepsilon$ **do**
 Résoudre le RMP pour obtenir $\hat{y}$;
 Mettre à jour LB ;
 Résoudre le sous-problème (SP) avec $\hat{y}$ fixé;
 if SP est non-réalisable **then**
 Générer une coupe de faisabilité (rayon extrême);
 Ajouter la coupe au RMP;
 else
 Mettre à jour UB ;
 Générer une coupe d'optimalité (point extrême);
 Ajouter la coupe au RMP;
return Solution courante

III Implémentation en C++

Pour l'implémentation de nos heuristiques, nous utilisons les fonctionnalités modernes du langage. Le code est indenté et coloré de manière sobre.

Listing A.1 – Ajout d'une coupe d'optimalité au solveur

```

1 #include <iostream>
2 #include <vector>
3
4 // Fonction permettant d'ajouter une coupe au modele
5 void addOptimalityCut(Model& masterProblem, const DualSolution& pi,
6     double objVal) {
7     Expr cutExpression;
8
9     // Construction de la coupe basique : theta >= pi^T * (b - B*y)
10    for (size_t i = 0; i < pi.size(); ++i) {
11        cutExpression += pi[i] * masterProblem.getSlack(i);
12    }
13
14    // Ajout effectif de la contrainte au probleme maitre
15    masterProblem.addConstraint( masterProblem.getTheta() >=
16        cutExpression );
17
18    std::cout << "Coupe ajoutee avec succes." << std::endl;
19 }
  
```

